Summary post

Following peer feedback and further reflection, I revisited my initial comparison of VeraCrypt and BitLocker and identified an important limitation in my earlier analysis: I had underemphasised usability as a primary security control. Zahid's feedback was particularly valuable in highlighting the need to consider both tool-specific weaknesses and practical mitigation strategies.

I therefore refine my original position. While I continue to recommend VeraCrypt for technically competent users, I now more clearly acknowledge the strengths of BitLocker. Its integration with the Windows ecosystem and Trusted Platform Module (TPM) reduces configuration complexity and, in doing so, mitigates user-induced vulnerabilities. This aligns with empirical findings that many real-world security failures arise from misconfiguration rather than from flaws in cryptographic design.

At the same time, I maintain that VeraCrypt offers stronger assurances in contexts where transparency and independent verification are required. Its open-source architecture enables external scrutiny and reduces reliance on vendor trust, with independent audits demonstrating how such transparency can expose implementation-level weaknesses. However, this model introduces governance considerations. VeraCrypt's development depends on a relatively small number of contributors, increasing the "bus factor" and creating reliance on individual maintainers. While transparency permits external review, the absence of a formally structured development process may constrain systematic peer review and continuous security auditing compared with institutionally supported systems. The historical discontinuation of TrueCrypt further illustrates how open-source encryption tools can become unsupported, leaving potential security issues unaddressed. In contrast, BitLocker benefits from structured internal development and predictable update cycles, but requires users to trust vendor-controlled vulnerability disclosure and patching processes.

Building on both peer input and further analysis, I extend my original ontology. In addition to Weakness, Trigger and Impact, usability should be treated as a contributing condition influencing the likelihood of insecure behaviour. In practice,

identical technical weaknesses may result in different risk levels depending on user capability and operational context. I further introduce lifecycle-oriented risks, including exit strategy and long-term data retention. Encrypted systems must support sustainable key management over extended periods, particularly where regulatory requirements mandate data retention for ten years or more. In such cases, loss of credentials or poorly managed backups can result in permanent data inaccessibility, independent of cryptographic strength.

This reflection has shifted my perspective from tool comparison towards a socio-technical evaluation of security posture. Secure software selection depends on the interaction between usability, transparency, governance and long-term operational risk, rather than on cryptographic strength considered in isolation.